

Southeast University

Lab Report 04

Forms, Validation, JSON and Local Storage

Course Title: Web and Internet Programming Lab

Course Code: CSE472



Submitted to

Mr Abid Ahmad

Faculty

Southeast University

Submitted by

Mst. Mahbuba Yasmin Maria

ID: 2022100010040

Department of Computer Science and Engineering

Southeast University

Date of Submission: _____

1. Objectives

- Read and validate Full Name, Student ID, Email Address and Workshop.
- Group the four values in one JavaScript object and convert it to JSON.
- Save, retrieve and clear one practice registration using localStorage.
- Test invalid input, valid input and persistence after a refresh.

2. Tools used

HTML, CSS and JavaScript; Google Chrome; Node.js and Playwright for automated browser checks, evidence and PDF export. Git/GitHub are required for submission but no repository publication was performed.

3. Work completed

The project copies the supplied Lab 03 event page into labs/lab-04-forms-json/. The event information, schedule, image and earlier interaction buttons are retained. The registration form is narrowed to the manual's four required fields. Register checks each value before constructing an object with name, studentId, email and workshop properties.

JSON.stringify() produces the text saved under the registration key. Show Saved Registration uses getItem() and JSON.parse() to display the saved values and JSON after refresh. Clear Saved Registration uses removeItem() and resets saved-data feedback. New registrations replace the previous item.

Small additions trim surrounding whitespace and show feedback when storage is blocked or saved JSON is unreadable. The original Lab 03 seat count of zero is retained as an independent demonstration. The lab does not connect seat counts to saved registrations and does not implement a server or database.

4. Implementation evidence — form

Workshop Registration

Enter practice details to save one registration in this browser. A new registration replaces the previous one.

Full Name

Show Greeting

Student ID

Email Address

Workshop

Register

Registration saved successfully.

Figure 1. Completed four-field form with a successful save. Nadia Rahman and the displayed Student ID are sample test values.

4. Implementation evidence — validation

Workshop Registration

Enter practice details to save one registration in this browser. A new registration replaces the previous one.

Full Name

Show Greeting

Student ID

Email Address

Workshop

Register

Please enter your full name.

Figure 2. An empty Full Name is rejected before JSON is created or saved.

4. Implementation evidence — JSON and refresh

JSON Output

```
{ "name": "Nadia Rahman", "studentId": "202312345", "email": "nadia@example.com", "workshop": "CSS Basics" }
```

Figure 3. Actual JSON output containing all four values, matching the stored text.

Saved Registration

Show Saved Registration

Clear Saved Registration

Nadia Rahman (Student ID: 202312345) registered for CSS Basics. Email: nadia@example.com.

Figure 4. After refreshing, Show Saved Registration reads the stored JSON and displays the student's details.

6. Key concept explanation

Concept	Use in this project
Form value reading	<code>getElementById()</code> finds the input; <code>.value</code> reads its current text. <code>trim()</code> removes surrounding spaces from text inputs.
Validation	Each empty string produces a specific message and <code>return</code> stops the save function.
Object	The registration object groups name, <code>studentId</code> , email and workshop as related properties.
<code>JSON.stringify()</code>	Converts the object into text for display and storage.
<code>JSON.parse()</code>	Converts saved JSON text back into an object so its properties can be read.
<code>localStorage</code>	<code>setItem</code> saves text, <code>getItem</code> reads it, and <code>removeItem</code> deletes only the registration key.

5. Representative code — validation and save

js/script.js — Read, validate and save

```
function submitRegistration() {
  let name = document.getElementById("studentName").value.trim();
  let studentId = document.getElementById("studentId").value.trim();
  let email = document.getElementById("studentEmail").value.trim();
  let workshop = document.getElementById("workshop").value;
  let message = document.getElementById("formMessage");

  if (name === "") {
    message.textContent = "Please enter your full name.";
    return;
  }
  if (studentId === "") {
    message.textContent = "Please enter your student ID.";
    return;
  }
  if (email === "") {
    message.textContent = "Please enter your email address.";
    return;
  }
  if (workshop === "") {
    message.textContent = "Please select a workshop.";
    return;
  }

  let registration = {
    name: name,
    studentId: studentId,
    email: email,
    workshop: workshop
  };
  let jsonData = JSON.stringify(registration);

  // Storage can be blocked by browser settings. Only confirm after saving.
  try {
    localStorage.setItem("registration", jsonData);
  } catch (error) {
    message.textContent = "Unable to save. Please allow browser storage and try again.";
    return;
  }

  document.getElementById("jsonOutput").textContent = jsonData;
  document.getElementById("savedMessage").textContent = "";
  message.textContent = "Registration saved successfully.";
}
```

Figure 5. Actual function: read four values, reject empty fields, create an object, stringify, save and confirm.

5. Representative code — load and clear

js/script.js — Load and clear

```
function showSavedRegistration() {
  let output = document.getElementById("savedMessage");
  let savedData;

  try {
    savedData = localStorage.getItem("registration");
  } catch (error) {
    output.textContent = "Unable to read saved data. Please allow browser storage and try again.";
    return;
  }

  if (savedData === null) {
    document.getElementById("jsonOutput").textContent = "No registration saved yet.";
    output.textContent = "No saved registration was found.";
    return;
  }

  // Handle damaged or unrelated data without crashing the page.
  try {
    let registration = JSON.parse(savedData);
    if (registration === null ||
        typeof registration.name !== "string" ||
        typeof registration.studentId !== "string" ||
        typeof registration.email !== "string" ||
        typeof registration.workshop !== "string") {
      output.textContent = "Saved data is not a registration. Clear it and register again.";
      return;
    }
    document.getElementById("jsonOutput").textContent = savedData;
    output.textContent = registration.name + " (Student ID: " + registration.studentId +
      ") registered for " + registration.workshop + ". Email: " + registration.email + ".";
  } catch (error) {
    output.textContent = "Saved data could not be read. Clear it and register again.";
  }
}

function clearRegistration() {
  let output = document.getElementById("savedMessage");
  try {
    localStorage.removeItem("registration");
  } catch (error) {
    output.textContent = "Unable to clear saved data. Please allow browser storage and try again.";
    return;
  }
  document.getElementById("jsonOutput").textContent = "No registration saved yet.";
  document.getElementById("formMessage").textContent = "";
  output.textContent = "Saved registration cleared.";
}
```

Figure 6. Actual saved-data functions, including missing data and storage-error feedback.

7. Testing and corrections

- Independent copied folder loads HTML, CSS, JavaScript and image, with matching labels and navigation.
- Show Saved Registration reports no saved item before registration.
- Empty name, Student ID, email and workshop each show the correct message without saving.
- Whitespace-only name and Student ID are rejected.
- Valid input saves all four exact values as JSON and displays the stored JSON.
- An invalid attempt does not overwrite an existing saved registration.
- Saved data survives refresh; Show Saved Registration parses it and restores the JSON display.
- A second registration replaces the first; text stays literal and leading zeros in Student ID are preserved.
- Clear removes only the registration key, clears stale feedback, and remains cleared after refresh.
- Invalid JSON and incompatible saved data show a recovery message without a page error.
- Simulated blocked storage reports save, read and clear failures without false success.
- Retained Lab 03 buttons work with the inherited zero-seat setting.
- 390px layout has no horizontal overflow; browser checks have no console or JavaScript errors.

The copied form originally used studentid and email IDs. These were changed to studentId and studentEmail, with matching labels, to follow the Lab 04 manual. The old server-style submit control was replaced by the Register button. No uncaught JavaScript errors occurred in the completed checks.

8. Conclusion and source information

The completed form validates four required values, groups them into an object and stores a JSON representation in the browser. Saved details can be retrieved after refresh or cleared using the dedicated button.

Limitations

The implementation uses basic presence checks. It stores one registration under the registration key; a subsequent valid save replaces the earlier record. This is browser-local storage, with no backend or database. The inherited Lab 3 seat count remains zero and is independent of the registration form.

Source files

lab-04-forms-json/index.html, css/style.css and js/script.js.

Assignment reference: CSE472_Lab_Manual_04_Forms_Validation_JSON_and_Local_Storage.docx used for the original implementation.

GitHub repository

<https://github.com/Mariya10040/Web-Internet-Programming-Lab>

Lab source: <https://github.com/Mariya10040/Web-Internet-Programming-Lab/tree/main/lab-04-forms-json>